

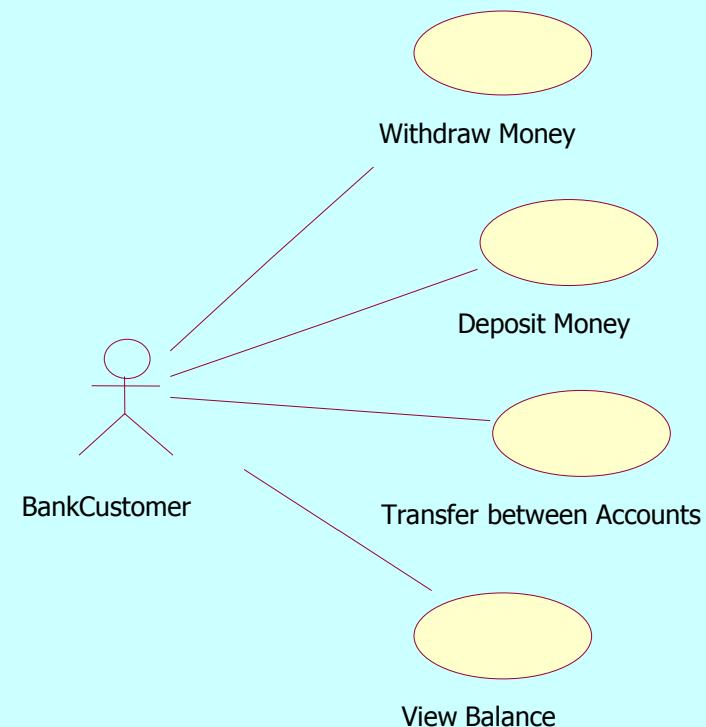
Use-case-driven development

- We illustrate the **use case driven development** approach (specific of the **Unified Process** [JBR99]) by means of a **case study**
 - An **Automated Teller Machine (ATM)** system based on examples from [JBR99,BB99,BB02,Kru03]
- In the **use-case driven** approach **each basic workflow produces a corresponding model:**
 - Use case model (developed in requirements workflow)
 - Analysis model
 - Design model
 - Implementation model
 - Test model

Case study: an ATM system

Use Case Model

- A **Use Case Model** is produced in the requirements workflow
- One can attach (natural language) descriptions to each use case, including descriptions of
 - Representative use case scenarios
 - E.g., the steps of a successful transfer between two accounts
 - Non-functional requirements (e.g., expected response time)
- The realization of the Withdraw Money use case is described in [JBR99]
- In this presentation we focus on the **Transfer between Accounts use case** realization
 - The other use cases must be handled in a similar manner

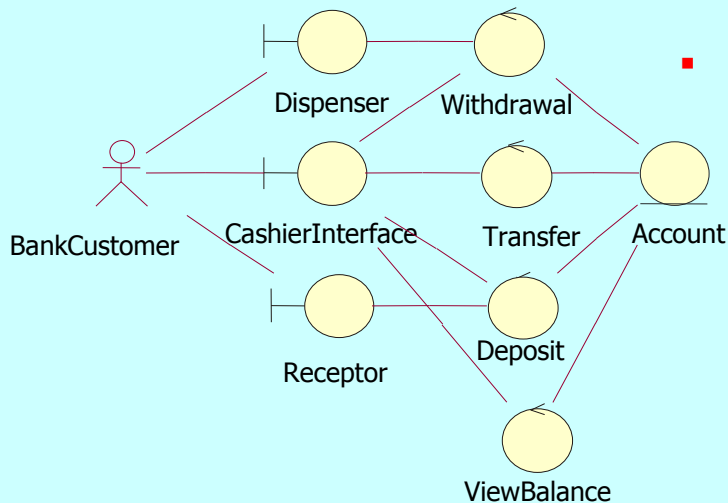


Case study: an ATM system

Analysis Model

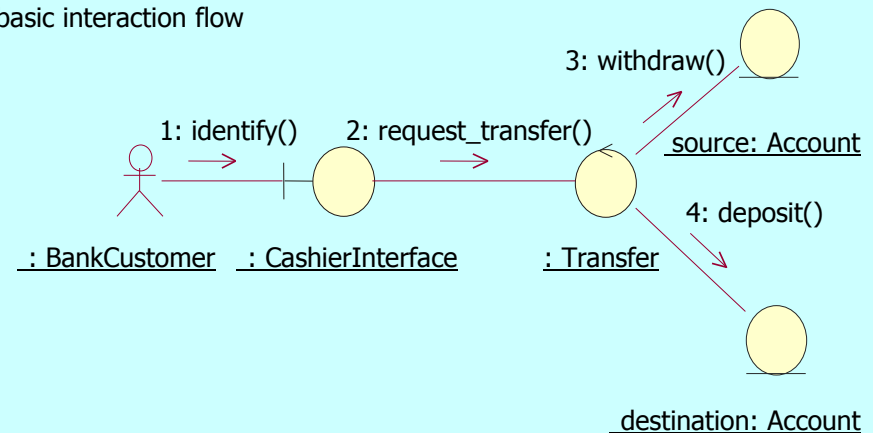
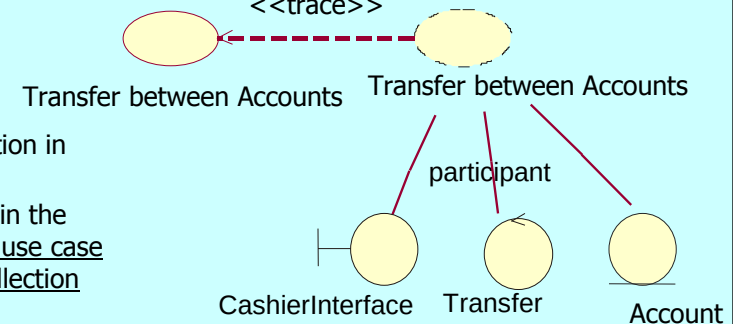
- The **Analysis Model** works as a first cut at the design model [JBR99]
- Engineers develop an analysis model in order to better understand the problem

- ATM Analysis structure
- There is one control class per use case realization



- Transfer between Accounts use case realization in the analysis model comprises
 - Analyses classes that participate in the realization of the use case (each use case is realized by a corresponding collection of analysis classes)
 - Communication (interaction) diagrams
 - E.g., for the basic interaction flow

Use Case Model <<trace>> Analysis Model



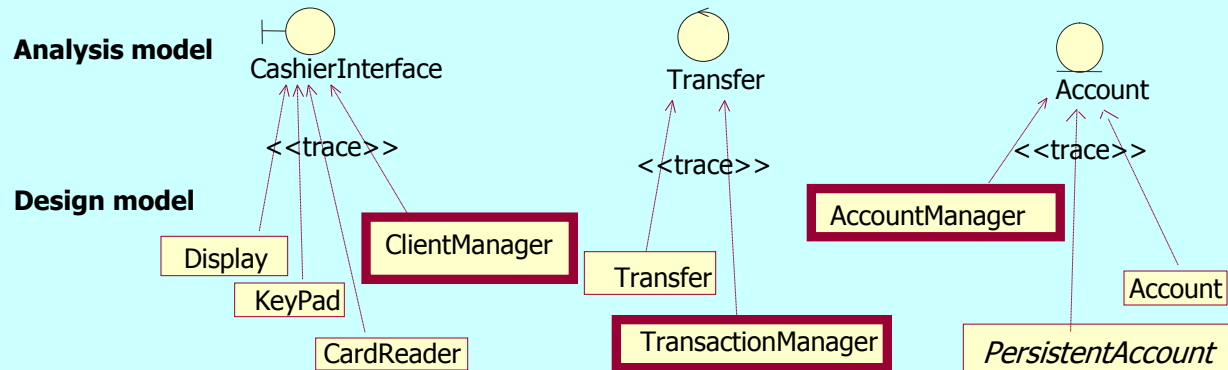
Case study: an ATM system

Design Model

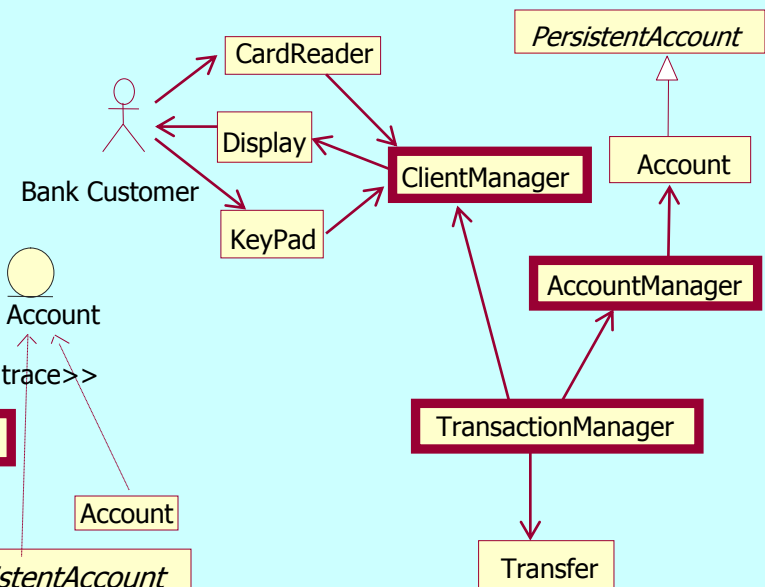
- The **Design Model** works as a blueprint for the implementation [JBR99]

- An analysis class is an abstraction of a number of design classes

- Design classes that participate in the realization of the Transfer between Accounts use case (tracing to corresponding analyses classes)



- Class diagram showing the relationships between the design classes that participate in the realization of the Transfer between Accounts use case



- ClientManager, TransactionManager, AccountManager are **active classes** [BRJ99] (designed to support a distributed implementation)
 - In general, a design class may participate in the design of several analysis classes; for example, ClientManager (TransactionManager) participates in the design of all interface (control) analysis classes
 - All the analyses classes specify and give rise to more refined design classes

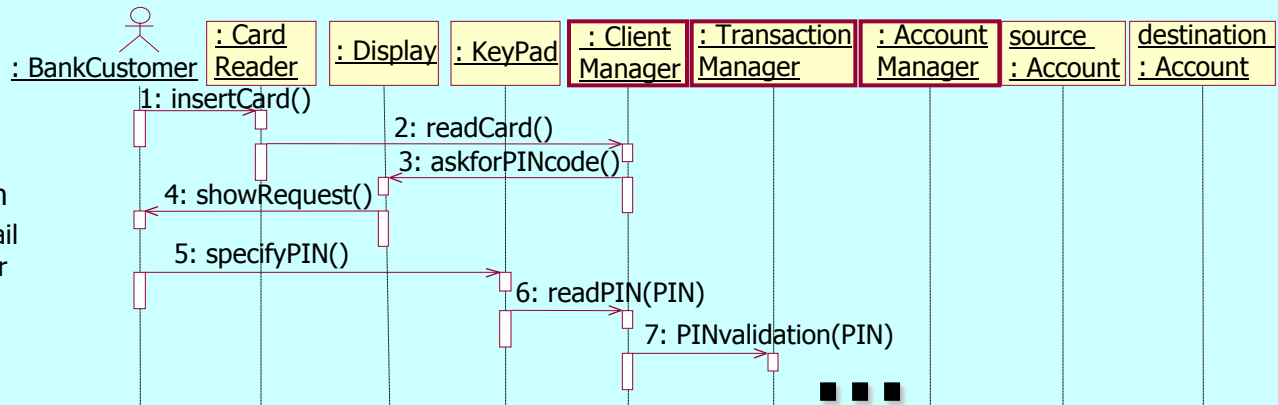
Case study: an ATM system

Design Model

Sequence Diagram

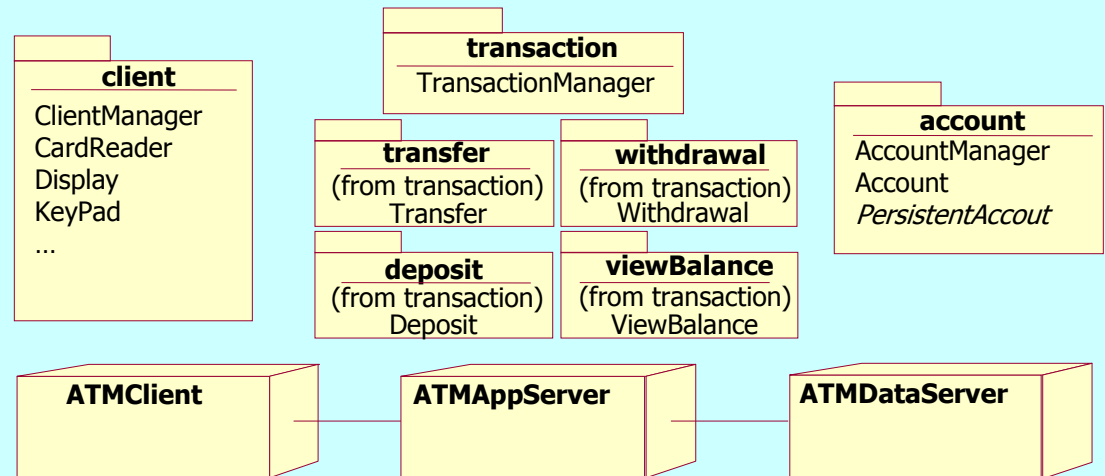
for the basic flow of the Transfer between Accounts use case realization in design

The amount of detail is significantly greater than in the (communication diagram from the) analysis model



Consider a software decomposition solution based on three main **packages** (subsystems): client, transaction, and account

The components that implement the three subsystems can be deployed on a **Three-Tier** architecture



Case study: an ATM system

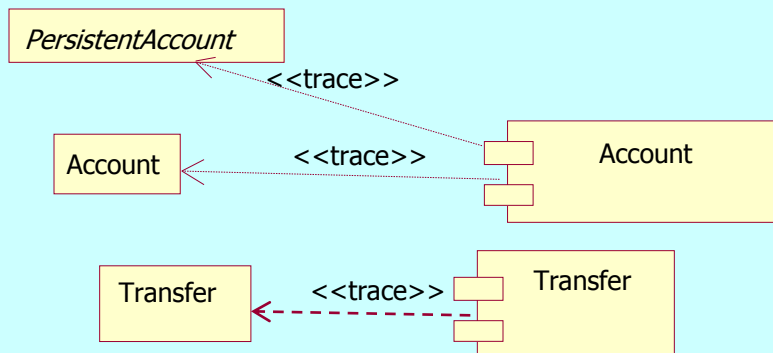
Implementation Model

- The implementation model shows **components** (that implement the design classes) and their dependencies

- (Some) Components that participate in the implementation of the Transfer between Accounts use case (tracing to design classes)

Design model

Implementation model



- You can also model compilation dependencies (to show which components are required to compile a specific component)

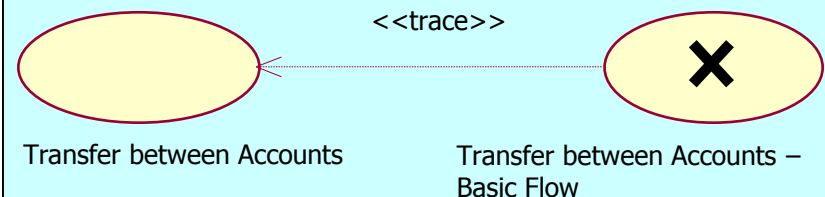
Test Model

- A **set of test cases** is derived **for each use case**

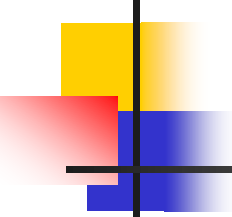
- Transfer between Accounts – Basic Flow test case (corresponds to Transfer between Accounts use case)

Use case model

Test model



- Each test case should include the following information [JBR99,LL05]:
 - What to test (identification information)
 - With which input
 - (Expected) result
 - Under which conditions



References

- [BB99] W. Boggs, M. Boggs. *Mastering UML with Rational Rose* (1st edition), Sybex, 1999.
- [BB02] W. Boggs, M. Boggs. *Mastering UML with Rational Rose* (2st edition), Sybex, 2002.
- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3rd edition). Addison-Wesley, 2003.
- [LL05] T. Lethbridge, R. Laganriere, *Object-Oriented Software Engineering: Practical Software Development using UML and Java*, 2nd edition, McGraw-Hill, 2005.
- [RS13] D. Rosenberg, M. Stephens, *Use Case Driven Object Modeling with UML: Theory and Practice*, Apress, 2013.
- [Som16] I. Sommerville. *Software Engineering*, (10th edition). Addison-Wesley, 2016.